순천향대학교

ARTIFICIAL INTELLIGENCE CONVERGENCE SOFTWARE
순천향대학교 인공지능 융합 소프트웨어 연구실

Momentum, RMSProp, Adam, Validation Data, K-fold cross Validation

박성진

발표 | 박성진 010 – 9450 – 6778 ppssjj020222@gmail.com 순천향대학교 멀티미디어관

CONTENTS

01 Momentum

02 RMSProp

03 Adam

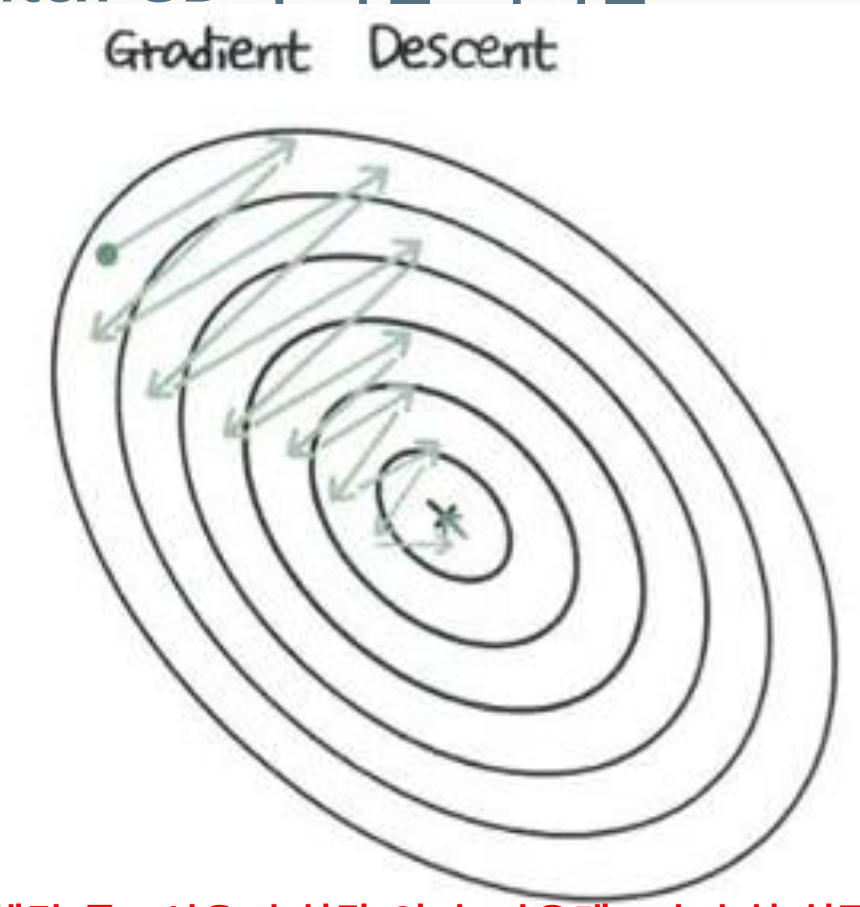
04 Validation Data

05 K-fold cross Validation

01 | Momentum 문제 정의

타원형 Loss함수의 등고선의 경우 GD, SGD, Mini-Batch GD의 학습 어려움

- 매 스텝에서 현재 기울기 방향으로만 이동
- 이런 길쭉한 골짜기에서는 기울기가 가파른 축 방향 성분이 크게 나옴
  - 골짜기가 좌우로 튕기듯 지그재그 발생
    - 손실 최솟점으로 진행이 더딤



- 방향을 매번 새로 정하니까, 골짜기 벽에 계속 부딪히면서 내려간다.

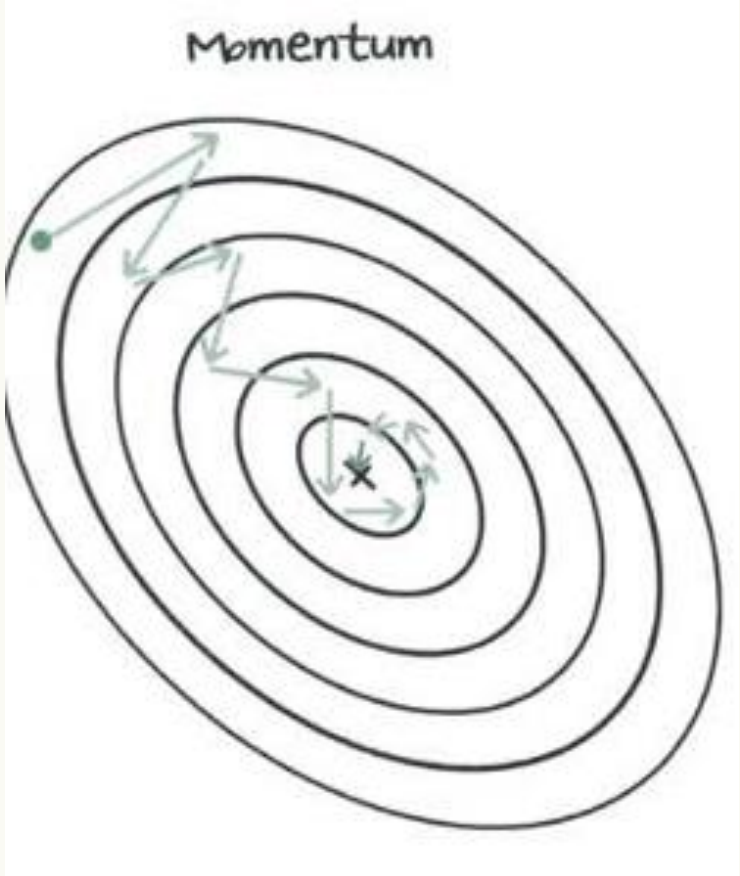
- 해당 등고선은 손실값 의미, 가운데 x가 손실 최적점

01 | Momentum

Momentum

Momentum – 기울기만 쓰는 것이 아니라, 이전 이동 방향을 누적하여 현재의 이동 방향을 결정(관성)

- 좌우로 흔들리는 성분은 감쇠
- 골짜기 아래로 가는 성분은 가속
- 경로가 더 매끈해지며 빠르게 수렴



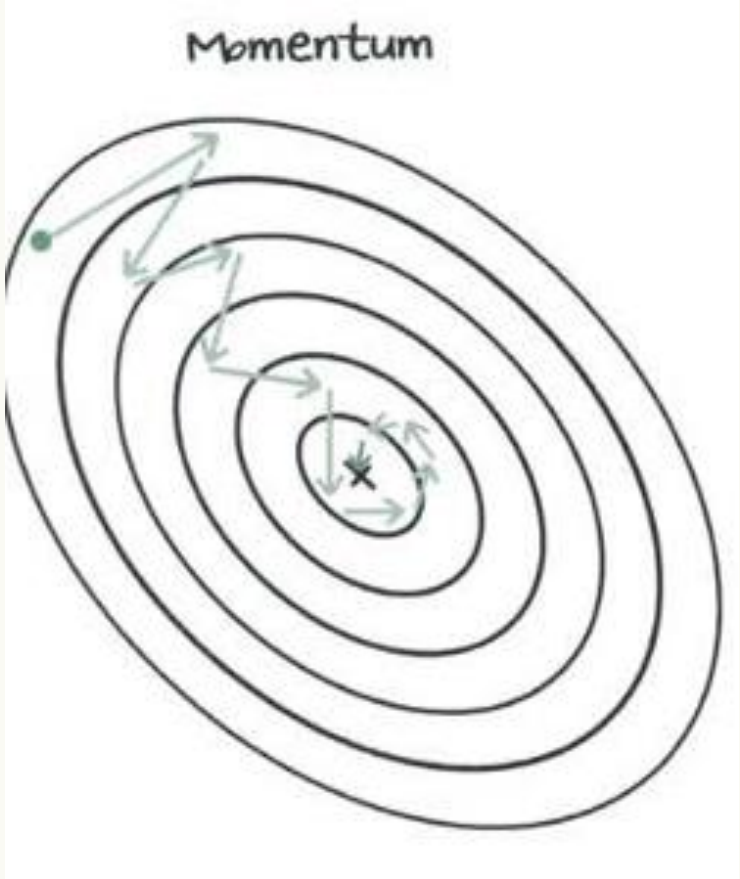
- 관성을 붙여서 흔들림을 줄이며, 한 방향으로 밀어준다.

- 해당 등고선은 손실값 의미, 가운데 x가 손실 최솟점

01 | Momentum 원리 파악

Momentum – 기울기만 쓰는 것이 아니라, 이전 이동 방향을 누적하여 현재의 이동 방향을 결정(관성)

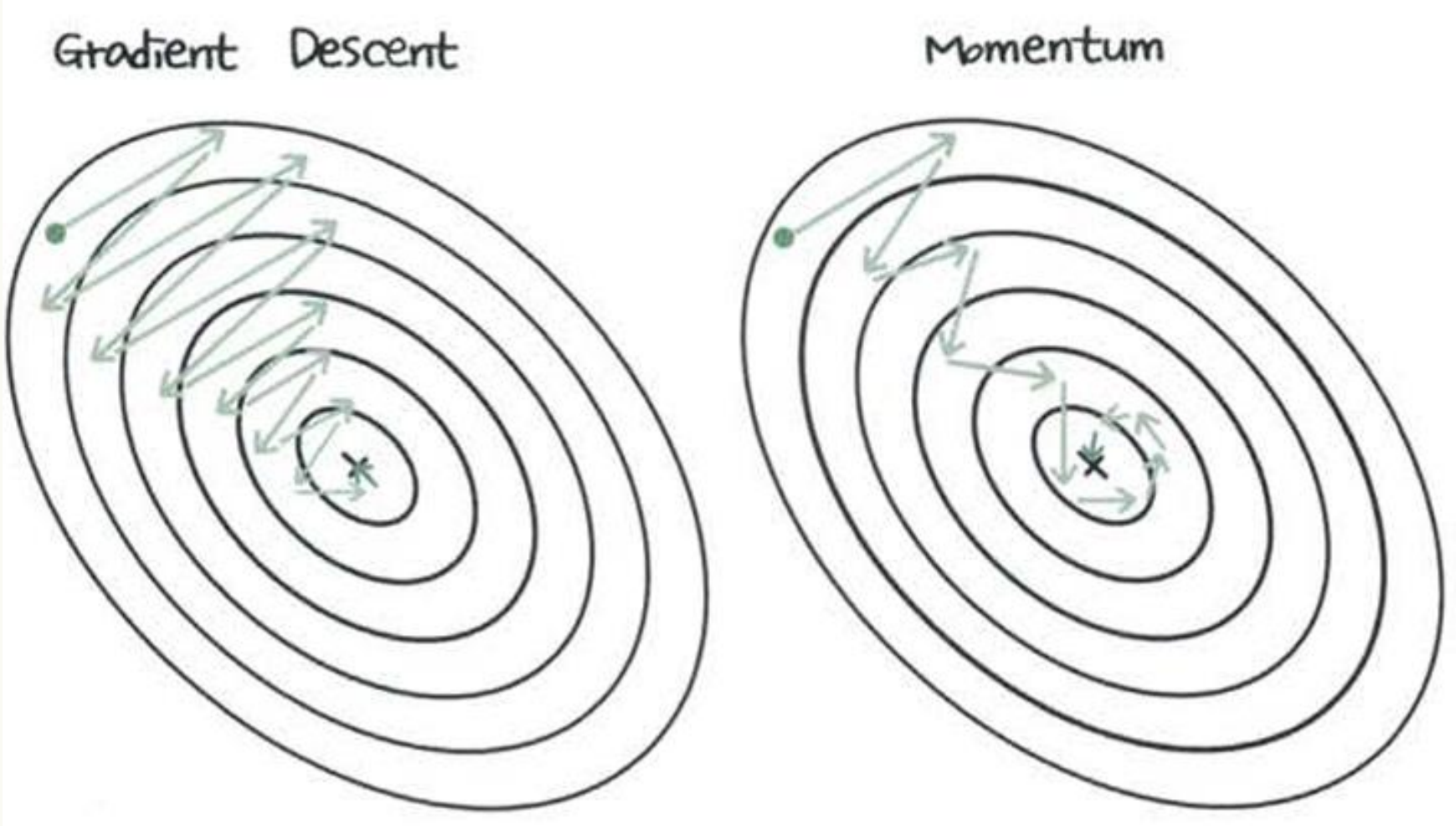
1. GD와 동일한 방향으로 이동
2. 현재와 이전 그래디언트를 **합산하여 방향을 결정.**
   1. 첫 이동이 왼쪽이었다면, 현재 그래디언트가 오른쪽을 가리켜도 이전의 왼쪽 방향의 **영향을** 받아 오른쪽으로의 이동을 **상쇄**
3. 현재 그래디언트가 왼쪽을 그리켜도 직전의 오른쪽 이동 관성이 남아 왼쪽의 이동이 **제한**



- 불필요한 좌우 진동 최소화, 최저점 수렴 가속화

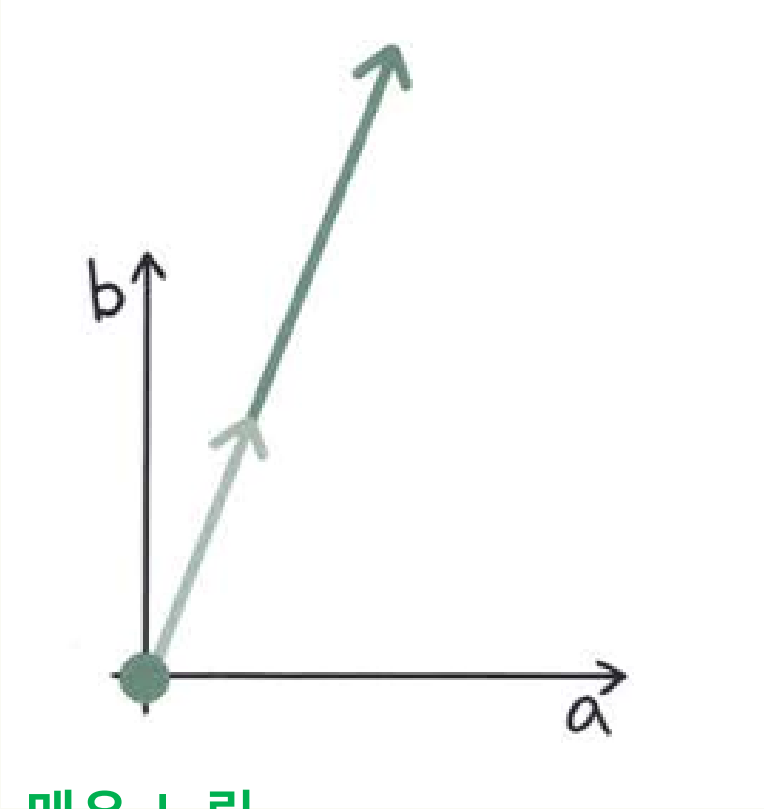
- 해당 등고선은 손실값 의미, 가운데 x가 손실 최솟점

01 | Momentum 비교



02 | RMSProp 문제 정의

- 파라미터 벡터 w = (a, b) = (0.1, 1)
- g=(0.1, 1) 처럼 **b축 기울기가 a축보다 10배 크다**
  - **$w_{t+1} = w_t - \eta g_t$** **η = 학습률**
    - **A는 0.1만큼 이동**
    - **B는 1만큼 이동**
    - **업데이트 시 b에서 훨씬 크게 이동**



η를 크게 잡으면: b축이 너무 크게 움직여 **오버슈팅/진동/발산** 위험
η를 작게 잡으면: b축은 안정적이지만 a축이 너무 조금 움직여 **진전이 매우 느림**

하나의 η로 두 축을 동시에 최적화하기가 어렵다.

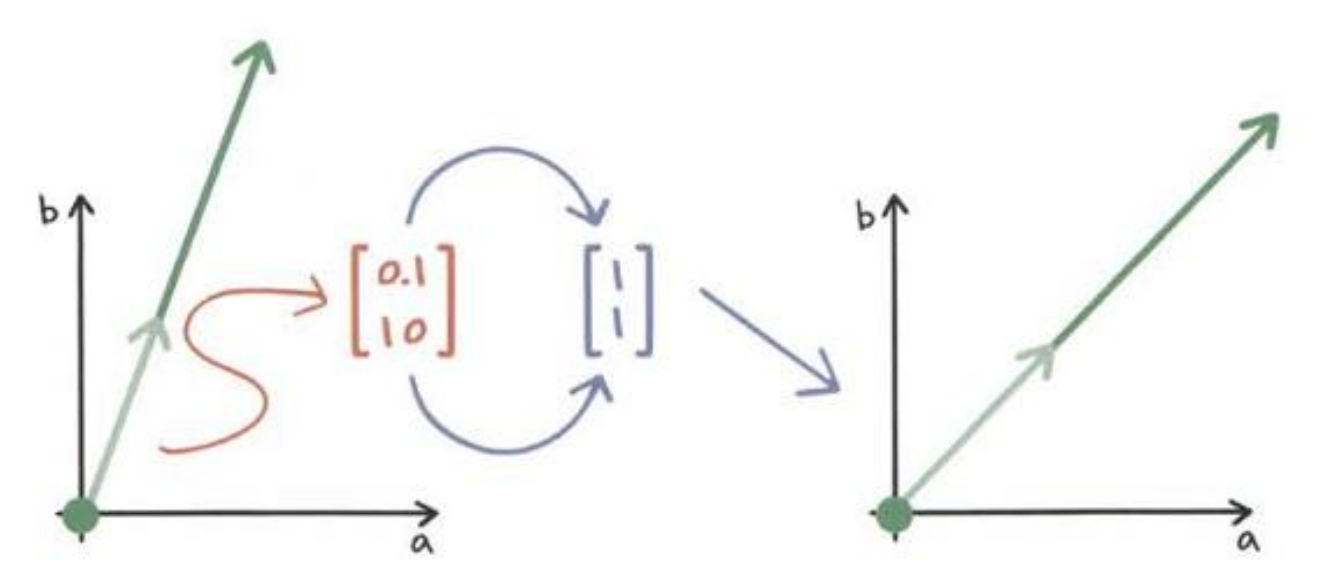
02 | RMSProp 원리 파악

RMSProp – 각 축(파라미터)의 최근 그래디언트 크기를 따로 관리

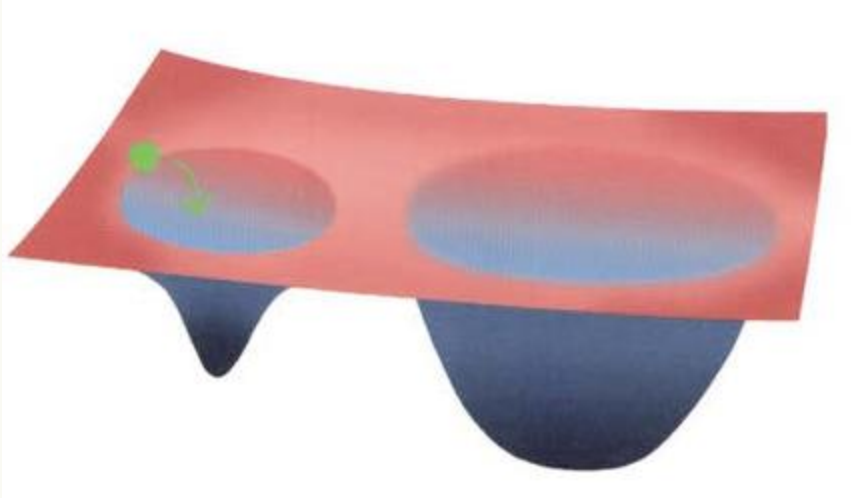
각 파라미터에 대한 편미분값을 제곱하여 누적

1. a, b에 대한 편미분 값(기울기)을 각각 제곱 g_a^2, g_b^2
2. 해당 제곱 값들을 누적(평균을 사용) $s_t = \rho s_{t-1} + (1-\rho) g_t^2$ 여기서 sa, sb가 각각 따로 쌓입니다.
3. 현재의 편미분값을 2.의 누적된 값의 제곱근 값으로 나누기 $\tilde{g}_t = \frac{g_t}{\sqrt{s_t} + \epsilon}$

- b축은 그래디언트가 계속 크면 sb가 커짐 → 분모 커짐 → **b축 스텝이 자동으로 작아짐**
- a축은 그래디언트가 상대적으로 작으면 sa가 작음 → 분모 작음 → **a축 스텝이 상대적으로 커짐**



02 | RMSProp 효과



가파른 골짜기와 완만한 평지 형태가 동시에 있는 Loss 함수

- 한쪽은 가파르고 민감한 축이라 조금만 움직여도 손실이 크게 바뀜
- 다른 쪽은 완만한 축이라 많이 움직여도 손실 변화가 작음

RMSProp는

- 가파른 축에서는 스텝을 줄여 오버슈팅과 진동을 자제
- 완만한 축에서는 스텝을 키워 평평한 구간을 빠르게 탈출

한 개의 학습률로 전체를 밀어붙이는 방식X
파라미터별로 학습률을 조절해 안정성과 속도를 동시에 확보

03 | Adam

문제 정의

Momentum

- 방향을 안정화하고 수렴을 빠르게
- 진동 감소, 수렴 가속

방향/속도(관성) 역할

RMSProp

- 파라미터별 스텝 크기 자동 조절 안정화
- 가파른 축에서 오버슈팅/발산 억제, 완만한 축에서 느린 진행 개선

스케일/분산(축별 크기) 역할

Adam : Momentum으로 방향을 만들고, RMSProp 스케일로 나눠서 업데이트

03 | Adam

원리 파악

$$\begin{bmatrix} a_{k+1} \\ b_{k+1} \end{bmatrix} = w_{k+1}, \quad \begin{bmatrix} a_k \\ b_k \end{bmatrix} = w_k, \quad \left. \begin{bmatrix} \frac{\partial L}{\partial a} \\ \frac{\partial L}{\partial b} \end{bmatrix} \right|_{a=a_k, b=b_k} = g_k$$

고,

$$\begin{bmatrix} a_{k+1} \\ b_{k+1} \end{bmatrix} = \begin{bmatrix} a_k \\ b_k \end{bmatrix} - \alpha \left. \begin{bmatrix} \frac{\partial L}{\partial a} \\ \frac{\partial L}{\partial b} \end{bmatrix} \right|_{a=a_k, b=b_k}$$

$$w_{t+1} = w_t - \eta g_t$$

03 | Adam

원리파악

$$w_{t+1} = w_t - \eta\, g_t$$

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

그래디언트 부분이 대체,
분자는 Momentum을 분모는 RMSProp을 반영

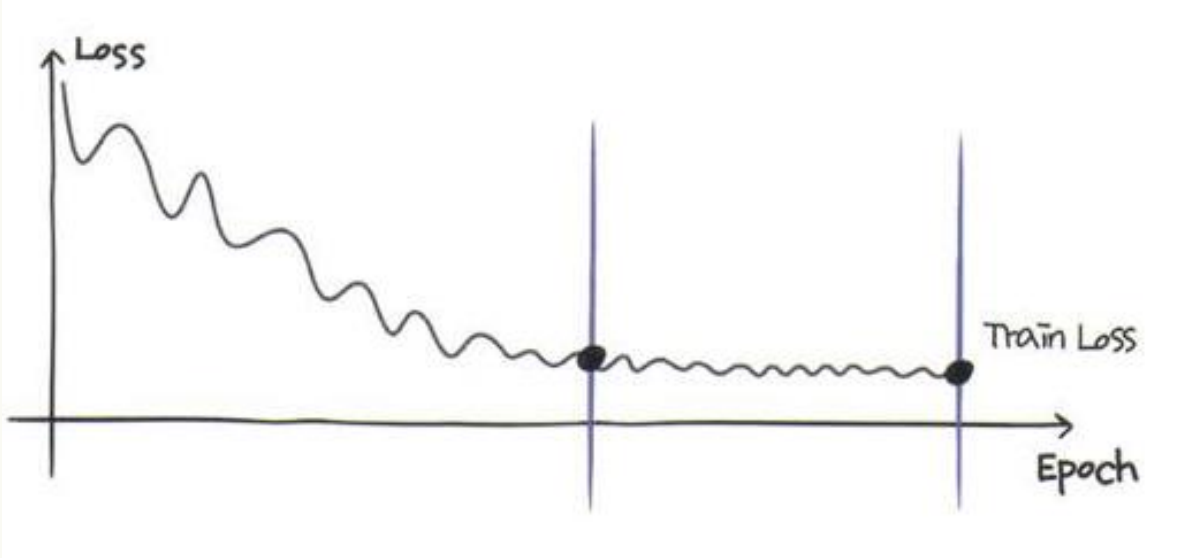
Require: α: Stepsize
Require: $\beta_1, \beta_2 \in$ 1/2 1/2): Exponential decay rates for the moment estimates
Require: $f(\theta)$: Stochastic objective function with parameters θ
Require: θ_0: Initial parameter vector
$m_0 \leftarrow 0$ (Initialize 1st moment vector)
$v_0 \leftarrow 0$ (Initialize 2nd moment vector)
$t \leftarrow 0$ (Initialize timestep)
while θ_t not converged **do**
$t \leftarrow t + 1$
$g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate) <- 방향 (exponential moving average)
$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate) <- 보폭
$\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
$\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ (Update parameters)
end while
return θ_t (Resulting parameters)

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- 방향은 관성을 가지게
- 보폭은 가파른 쪽 은 조심조심.. 완만한 쪽은 과감하게! (평준화를 통해 공평하게 탐색)

04 | Validation Data 문제 정의

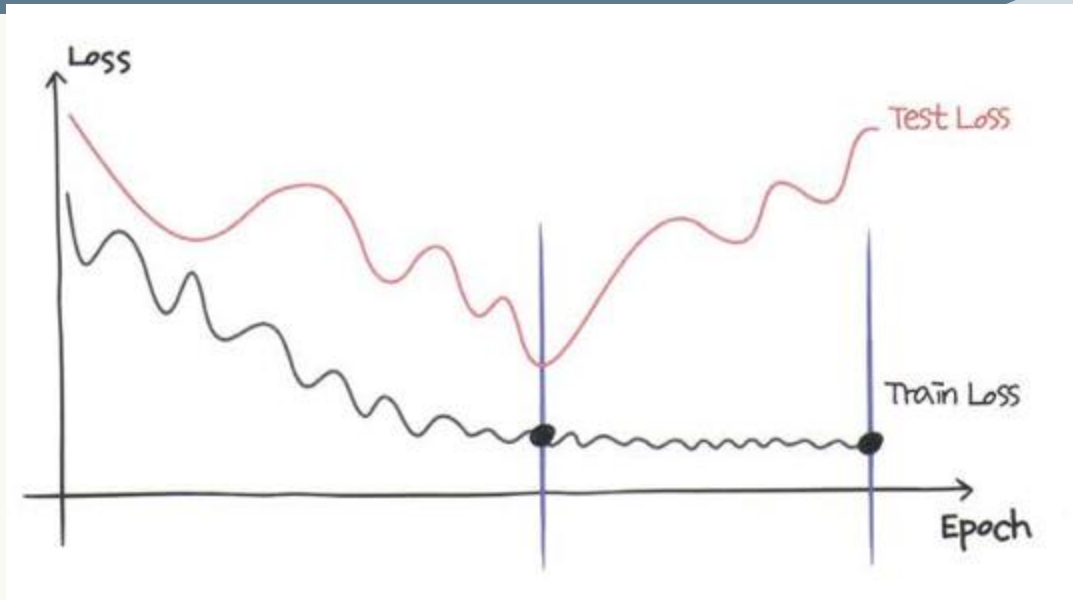
AI의 진정한 목표는 훈련 데이터가 아닌 테스트 데이터(실제)에서 좋은 성능을 보이는 것



- 딜레마 발생
  - 테스트 데이터를 학습 과정에서 사용해버리면 실제에서의 성능 파악 불가
  - 훈련 데이터만으로는 언제 학습을 멈춰야할지 판단의 어려움

04 | Validation Data

문제 정의



- 딜레마 발생
  - 테스트 데이터를 학습 과정에서 사용해버리면 실제에서의 성능 파악 불가
  - 훈련 데이터만으로는 언제 학습을 멈춰야할지 판단의 어려움

- 테스트 데이터를 기준으로 학습을 조정하면, 그 특정 데이터에 과적합된 모델을 얻게됨

04 | Validation Data 문제 정의

- 처음엔 정답을 알려주면서 훈련, 그 다음엔 처음보는 데이터를 보여주면서 잘하나 테스트

- 일단, Test 데이터가 학습 때 사용되면 안됨. Why?
  - AI가 처음 보는 data에 대해서도 잘하는지를 봐야하기 때문!
  - (즉, test data에 대해서 잘 하는게 AI의 목표!(실전에 강한!))

- 그런데, train 데이터만으로 학습하자니 몇 epoch에서 학습을 멈춰야할지 모른다
  - Train loss를 계속 줄이는게 능사가 아님.. 그래서 validation data
    (training 데이터의 일부를 validation 데이터로 삼고 기울기 구하는 데에는 참여 x)

04 | Validation Data 원리 파악

검증데이터 : 훈련 데이터의 일부를 떼어내 Epoch, Batch Size등의 하이퍼파라미터 조정

훈련 데이터 → 연습 문제 → 파라미터 학습(공부)

검증 데이터 → 모의고사 문제 → 하이퍼파라미터 결정(하루 공부 시간, 수면 시간, 공부 방법)

테스트 데이터 → 수능 문제 → 성능 측정(수능 성적 확인)

04 | Validation Data 원리 파악

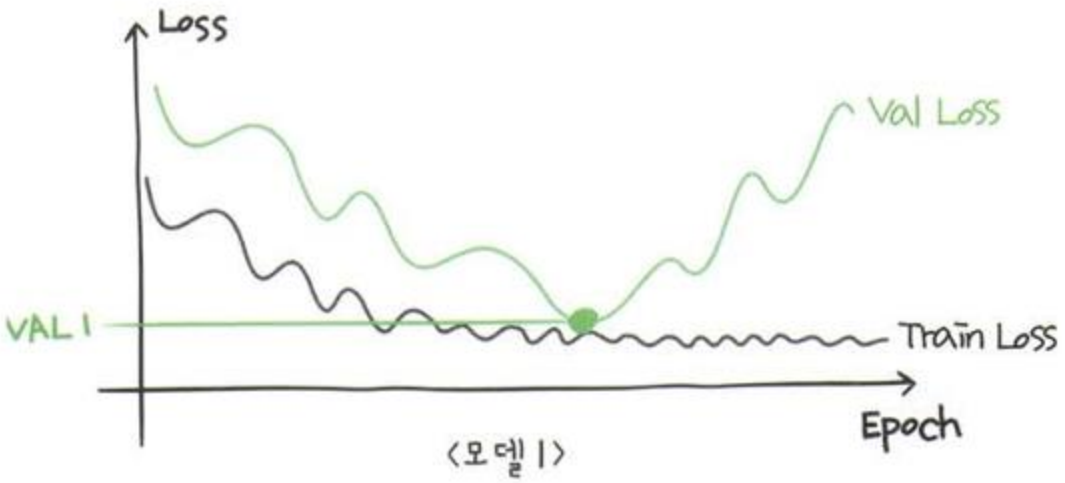
모델 구조 선택(하이퍼 파라미터)에서도 활용

모델 1 (빨리 오버피팅)

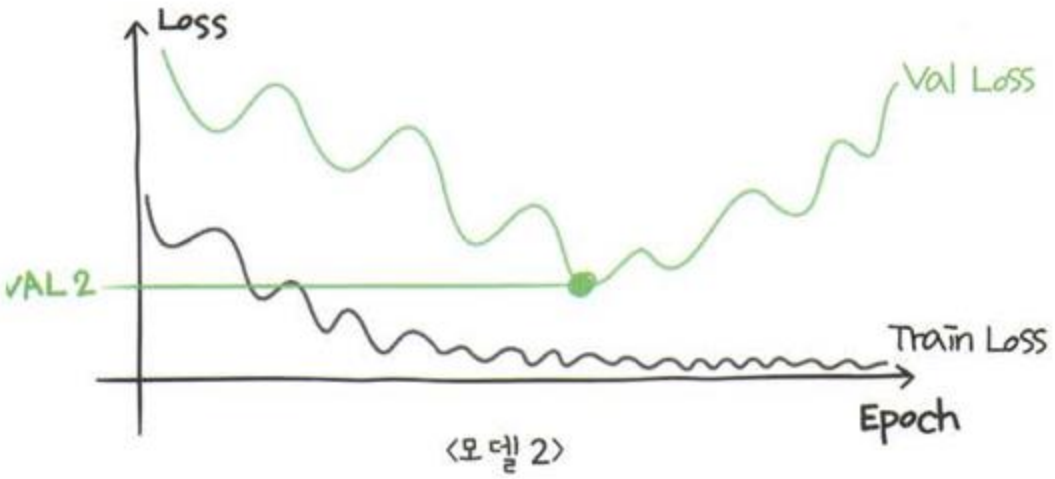
- 구조: Hidden 3층
- 설명 : Train Loss는 계속 감소 하지만 Val loss는 오버피팅

모델 2 (일반화 좋음)

- 구조: Hidden 1층
- 설명 : Train Loss 하강은 조금 느리지만, Val loss최저점이 더 안정적이고 오버피팅이 늦음



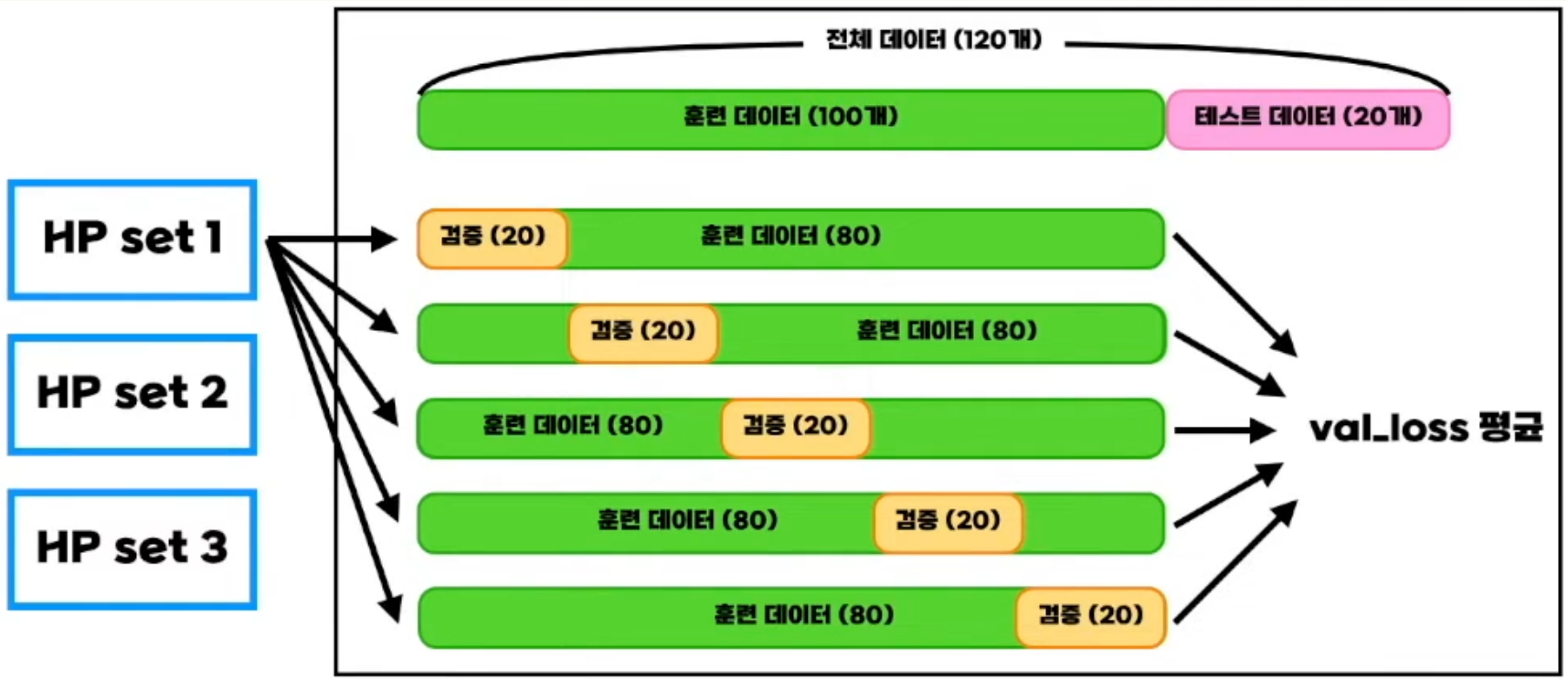
<모델 1>



<모델 2>

05 | K-fold Validation 연구 배경

- 전체 데이터 120개
- 훈련 데이터 100개
- 검증 데이터 20개
- 만약 강아지, 고양이 를 구분하는 문제라면 만약! 모든 검증 데이터가 강아지라면!!
  - 하이퍼파라미터는 강아지 사진에 편향된 모델을 만든다.



- Training data 가 적어서 일부를 쓰기 곤란할 때.. (5-fold로)
- 왜 곤란? 편향된 validation 데이터 => 각기 다른 train, validation조합의 데이터로 5개 모델을 만들자
- 평균 val loss를 구하자! => 가장 loss평균이 작은 hyperparameter set을 고르는데 사용 가능
- 선택된 set으로 training data 전체에 대해서 새롭게 학습시키거나,
  학습했던 5개 모델의 출력 결과를 하나로 합치기도 함 (majority vote, soft vote)